The classic perceptron uses a hard threshold, usually the **Heaviside** step function:

$$\hat{y} = f(z) = \begin{cases} 1 \text{ if } z \geq 0 \\ 0 \text{ if } z < 0 \end{cases}$$

If using $\{-1, +1\}$ labels instead:

$$\hat{y} = \text{sign}(z)$$

For a single training example, the weight update rule is:

$$w_i \leftarrow w_i + \eta e x_i$$

and the bias update is:

$$b \leftarrow b + \eta e$$

with

$$e = y - \hat{y}$$

notice there are only 2 possible values for e:

$$e = \begin{cases} 1 \text{ if } (y, \hat{y}) = (1, 0) \\ -1 \text{ if } (y, \hat{y}) = (0, 1) \end{cases}$$

what do either of these cases mean? Well, if it's 1, that means that output was negative when it was supposed to be positive. And if it was −1 that means that the output was positive when it was supposed to be negative. So what are we doing multiplying this e by the input value to 'train'? Let's look at how we compute our output in more detail:

$$z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$$

we are multiplying each weight by the *value of the input* x_i. So if the input is positive and we want the output to be positive then we want $x_i w_i$ to be positive! So the weight should also be positive! And so we need to converge to that positive weight if it happens to be negative which is why we add $x_i e$ to the weight. If the result was positive